Сортировка выбором

Степанов Николай Викторович 151319

Code and Cold Club 2026

Оглавление

1. Память компьютера и способы хранения данных
2. Как работает память
3. Адрес ячейки памяти
4. Массивы и связанные списки
5. Список задач
6. Массивы
7. Непрерывное хранение
8. Проблема добавления
9. Перемещение в новую область
10. Бронирование места
11. Связанные списки
12. Произвольное размещение
13. Указатели
14. Как в игре «Найди клад»
15. Вставка в середину списка
16. Список решает задачу
17. Массив требует сдвига
18. Сравнение массивов и связанных списков: доступ, кэширование и память
19. Произвольный и последовательный доступ
20. Массивы и произвольный доступ
21. Кэширование
22. Расход памяти
23. Сводная таблица сложностей
24. Что используется чаще?
25. Сортировка выбором
26. Постановка задачи
27. Идея сортировки выбором
28. Продолжение
29. Оценка $O(n^2)$
30. Уменьшение количества проверяемых элементов
31. Перспектива быстрой сортировки
32. Пример кода и шпаргалка
33. Поиск наименьшего элемента
34. Сортировка выбором
35. Практикум
36. Задание
37. Learn X in Y minutes
38. Обратная связь
39. Источники

Память компьютера и способы хранения данных

Как работает память

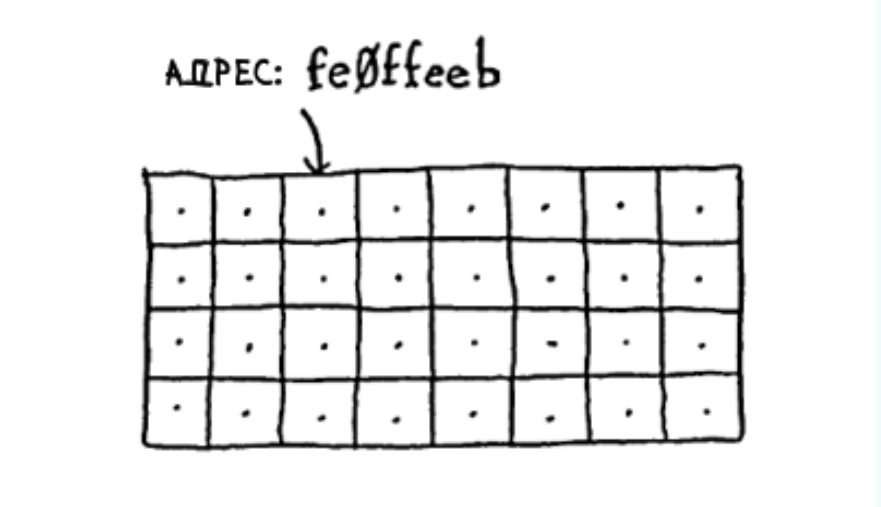
Представьте, что вы пришли в театр и хотите оставить свои личные вещи в гардеробе. Для хранения вещей есть специальные ящики.

В каждом ящике помещается один предмет. Вы хотите сдать на хранение две вещи, поэтому требуете выделить вам два ящика.



Адрес ячейки памяти

В сущности, именно так работает память вашего компьютера. Она представляет собой нечто вроде огромного шкафа со множеством ящиков, и у каждого ящика есть адрес.



Массивы и связанные списки

Каждый раз, когда вы хотите сохранить в памяти отдельное значение, вы запрашиваете у компьютера место в памяти, а он выдает адрес для сохранения значения.

Если же вам понадобится сохранить несколько элементов, это можно сделать двумя основными способами: воспользоваться массивом или списком.

Не существует единственно верного способа сохранения данных на все случаи жизни, поэтому вы должны знать, в чем отличие разных способов.

Список задач

Иногда в памяти требуется сохранить список элементов. Предположим, вы пишете приложение для управления текущими делами. Описания задач должны храниться в виде списка в памяти.

Что использовать — массив или связанный список?



Массивы

Непрерывное хранение

При использовании массива все задачи хранятся в памяти непрерывно (то есть рядом друг с другом).



Проблема добавления

Теперь предположим, что вы захотели добавить четвертую задачу. Но следующий ящик уже занят — там лежат чужие вещи!



Перемещение в новую область

Представьте, что вы пошли в кино с друзьями и нашли места для своей компании, но тут приходит еще один друг, и ему сесть уже некуда. Приходится искать новое место, где смогут разместиться все.

В нашем случае вам придется запросить у компьютера другой блок памяти, в котором поместятся все четыре задачи, а потом переместить все свои задачи туда.

Бронирование места

Если вдруг придет еще один друг, места опять не хватит и вам придется перемещаться снова! Сплошная суета. Кроме того, добавление новых элементов в массив станет серьезной проблемой.

Простейшее решение — «бронирование мест»: даже если список состоит всего из 3 задач, вы запрашиваете у компьютера места на 10 позиций… просто на всякий случай.

- Лишнее место может не понадобиться, и тогда память будет расходоваться неэффективно.
- Если в список будет добавлено более 10 задач, перемещать все равно придется.

В общем, прием неплохой, но его нельзя назвать идеальным.

Связанные списки

Произвольное размещение

При использовании связанного списка элементы могут размещаться где угодно в памяти.



Указатели

В каждом элементе хранится адрес следующего элемента списка. Таким образом, набор произвольных адресов памяти объединяется в цепочку.



Связанные адреса памяти

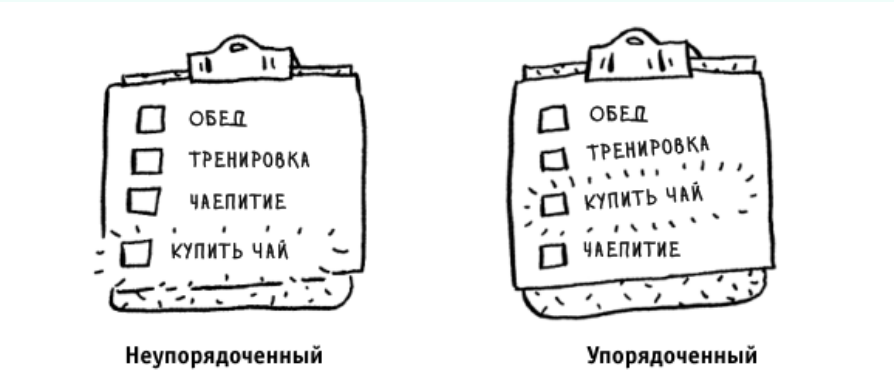
Как в игре «Найди клад»

Все как в игре «Найди клад». Вы приходите по первому адресу, там написано: «Следующий элемент находится по адресу 123». Вы идете по адресу 123, там написано: «Следующий элемент находится по адресу 847» и т. д.

Добавить новый элемент в связанный список проще простого: просто разместите его по любому адресу памяти и сохраните этот адрес в предыдущем элементе.

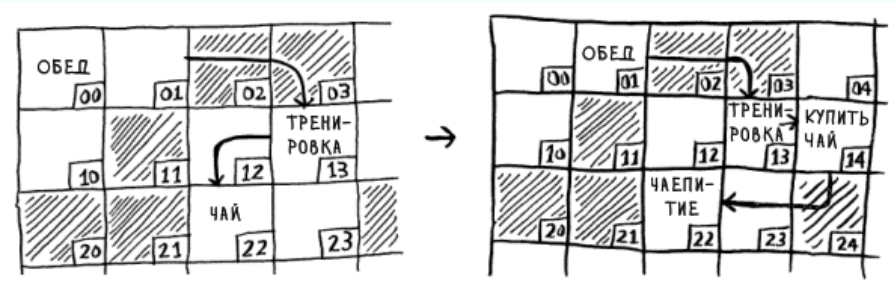
Вставка в середину списка

Предположим, вы решили, что список задач должен больше напоминать календарь. Прежде данные добавлялись только в конец списка, а теперь они должны добавляться в порядке их выполнения.



Список решает задачу

Со списком задача решается изменением указателя в предыдущем элементе.



Массив требует сдвига

А при работе с массивом придется сдвигать все остальные элементы.

А если свободного места не осталось, все данные придется скопировать в новую область памяти! В общем, списки лучше подходят для вставки элементов в середину.



Сравнение массивов и связанных списков: доступ, кэширование и память

Произвольный и последовательный доступ

Всего существуют два вида доступа: произвольный и последовательный. При последовательном доступе элементы читаются по одному, начиная с первого.

Связанные списки поддерживают только последовательный доступ. Чтобы прочитать 10-й элемент связанного списка, вам придется прочитать первые 9 элементов и перейти по ссылкам к 10-му элементу.

Массивы и произвольный доступ

адрес каждого его элемента. Допустим, массив содержит пять элементов и вы знаете, что он начинается с адреса 00. По какому адресу хранится пятый элемент?

Простейшая математика дает ответ: это адрес 04. Массивы прекрасно подходят для чтения элементов в произвольных позициях, потому что обращение к любому элементу в массиве происходит



Кэширование

Даже безотносительно произвольного доступа массивы работают быстрее, поскольку могут использовать кэширование.

Компьютеры читают целыми разделами, поскольку это значительно ускоряет переход к следующему элементу.

С массивами можно прочитать целый раздел элементов. В связанном списке местоположение следующего элемента неизвестно. Придется прочитать элемент, узнать, где находится следующий, а затем прочитать и его.



Расход памяти

Помните, я говорил, что в случае с массивами обычно выделяют больше памяти, чем нужно, и если в конечном итоге она не задействуется, то ее ресурс расходуется впустую?



На самом деле объем такой неиспользованной памяти получается не очень большим. С другой стороны, когда вы применяете связанный список, вы выделяете дополнительную память для каждого элемента, поскольку в ней сохраняется адрес следующего элемента.



Сводная таблица сложностей

	Массивы	Списки
Чтение	O(1)	O(n)
Вставка	O(n)	O(1)
Удаление	O(n)	O(1)

O(n) = линейное время

O(1) = постоянное время

Что используется чаще?

Массивы используются чаще, поскольку имеют множество преимуществ перед связанными списками. Во-первых, их проще читать. Во-вторых, массивы поддерживают произвольный доступ.

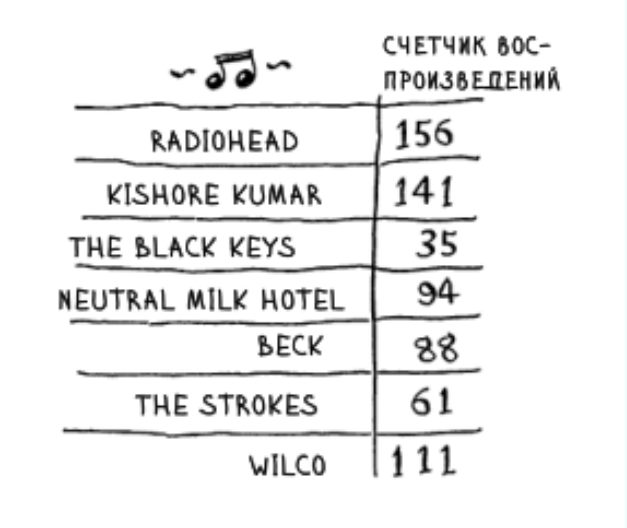
Таким образом, массивы используются чаще, чем связанные списки, за исключением особых случаев.

Сортировка выбором

Постановка задачи

А теперь объединим все, что вы узнали, во втором алгоритме: сортировке выбором. Чтобы освоить этот алгоритм, вы должны понимать, как работают массивы и списки и «О-большое».

Допустим, у вас на компьютере записана музыка и для каждого исполнителя хранится счетчик воспроизведений.



Идея сортировки выбором

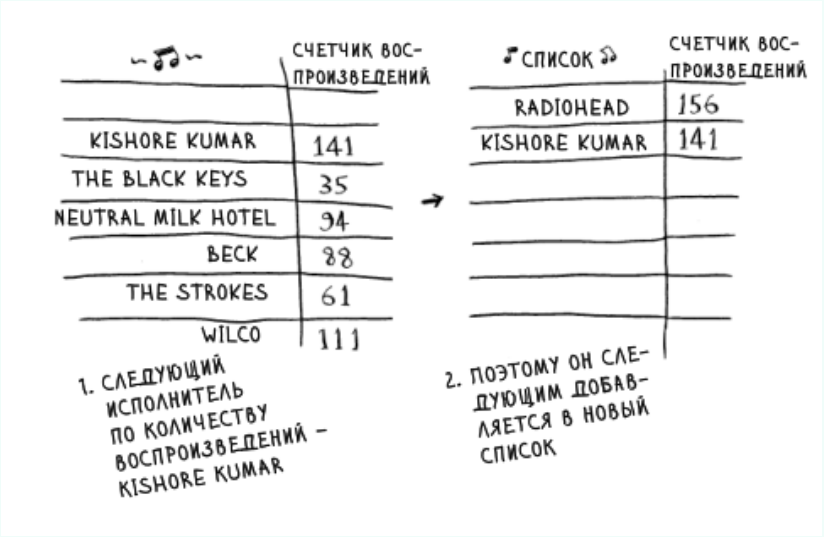
Вы хотите отсортировать список по убыванию счетчика воспроизведений, чтобы самые любимые исполнители стояли на первых местах.

Одно из возможных решений — пройти по списку и найти исполнителя с наибольшим количеством воспроизведений. Этот исполнитель добавляется в новый список.



Продолжение

Потом то же самое происходит со следующим по количеству воспроизведений исполнителем.



Продолжая действовать так, мы получаем отсортированный список.

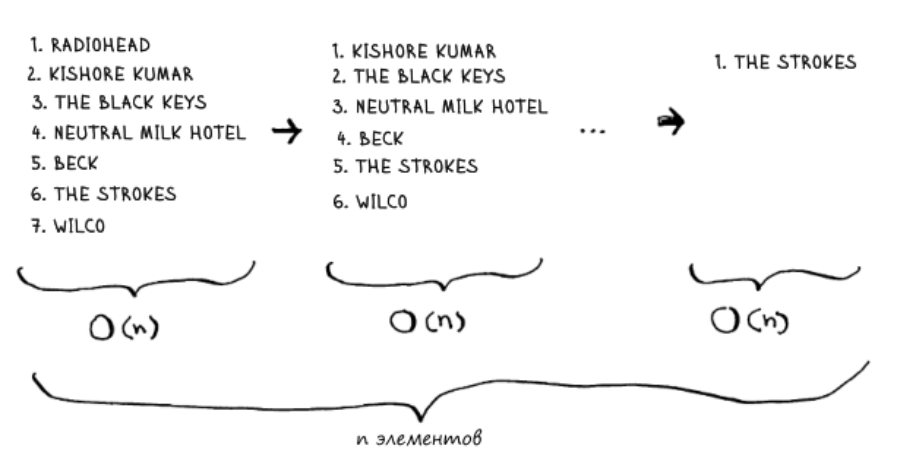


Оценка O(n²)

Чтобы найти исполнителя с наибольшим значением счетчика воспроизведения, необходимо проверить каждый элемент в списке. Это делается за время O(n).

Итак, имеется операция, выполняемая за время O(n), и ее необходимо выполнить n раз.

Все это требует времени O(n × n), или O(n²).



Уменьшение количества проверяемых элементов

При каждом выполнении операций количество элементов, которые нужно проверить, сокращается. Со временем все сведется к проверке всего одного элемента.

Почему же время выполнения все равно оценивается как $O(n^2)$? Сначала проверяются n элементов, потом n – 1, n – 2 … 2, 1. В среднем проверяется список из ½ × n элементов. Однако константы (такие, как ½) в «О-большом» игнорируются, поэтому мы просто используем $O(n \times n)$, или $O(n^2)$.

Перспектива быстрой сортировки

Алгоритм сортировки выбором легко объясняется, но медленно работает.

Быстрая сортировка — эффективный алгоритм сортировки, который выполняется за время O(n log n). Но мы займемся этой темой в следующий раз!

Пример кода и шпаргалка

Поиск наименьшего элемента

PYTHON

```python
def findSmallest(arr):
    smallest = arr[0]  # Для хранения наименьшего значения
    smallest_index = 0 # Для хранения индекса наименьшего значения

    for i in range(1, len(arr)):
        if arr[i] < smallest:
            smallest = arr[i]
            smallest_index = i

    return smallest_index
```

Сортировка выбором

PYTHON

```python
def selectionSort(arr):
    newArr = []

    for i in range(len(arr)):
        # Находит наименьший элемент в массиве и добавляет его в новый массив:
        smallest = findSmallest(arr)
        newArr.append(arr.pop(smallest))

    return newArr

print(selectionSort([5, 3, 6, 2, 10])) # Сортирует массив
```

Практикум

Задание

1. Откройте папку `problems/L0` на рабочем столе.
2. Откройте `task.*.md` и прочитайте алгоритмическую задачу.
3. Запишите своё решение в файл `main.py`.
4. Проверьте решение:
    - Откройте терминал `konsole`.
    - Напишите `python .`
    - Перетащите `test.py` в окно терминала; запустите.

Для помощи с синтаксисом python см. офф. документацию или следующий слайд.

Learn X in Y minutes

Краткий референс по Python

https://learnxinyminutes.com/python/

Click to open in browser

Обратная связь

Выразите своё мнение, идеи и пожелания в форме обратной связи

https://forms.yandex.ru/cloud/6af0d583ba5f02cec16bfb95

Click to open in browser

Источники

- Бхаргава, Адитья. Грокаем алгоритмы: 2-е издание. Питер, 2024.